

Clasificación de dígitos MNIST: comparación entre ANN y CNN

Resultado principal. La red neuronal convolucional (CNN) fue el mejor enfoque: alcanzó 98.89% de exactitud y 98.88% de F1 macro en 8,400 imágenes de prueba. Superó a la red neuronal artificial (ANN) en 1.76 y 1.78 puntos porcentuales, respectivamente, y redujo los errores de 241 a 93.

Objetivo y datos

Se clasificaron dígitos manuscritos del 0 al 9 usando `train.csv`: 42,000 observaciones, una etiqueta y 784 intensidades de píxel. No se detectaron valores faltantes; los píxeles se encontraron en el rango esperado de 0 a 255.

Los datos se dividieron de forma estratificada: 33,600 imágenes (80%) para entrenamiento y 8,400 (20%) para prueba, con semilla 42. El conjunto de prueba se mantuvo intacto hasta la evaluación final.

Preprocesamiento y entrada

Cada fila se normalizó a `[0,1]` y se reconstruyó mediante `reshape` como una imagen **28×28×1**. La CNN recibió directamente estas matrices y conservó la vecindad espacial. La ANN recibió las mismas imágenes, pero una capa `Flatten` las convirtió internamente en vectores de 784 elementos.

Esta diferencia implementa la indicación de la clase 05: la red convolucional opera con la imagen bidimensional, mientras la red artificial requiere una representación aplanada.

Arquitecturas y entrenamiento

ANN (109,386 parámetros)
`Flatten → Dense(128, ReLU) → Dropout(0.20) → Dense(64, ReLU) → Dropout(0.20) → Dense(10, Softmax)`.

CNN (421,642 parámetros)
`Conv2D(32, 3×3) → MaxPool(2×2) → Conv2D(64, 3×3) → MaxPool(2×2) → Flatten → Dense(128, ReLU) → Dropout(0.30) → Dense(10, Softmax)`.

Ambas redes se implementaron con TensorFlow/Keras, optimizador Adam, entropía cruzada categórica dispersa, lotes de 128, hasta 10 épocas y 10% de validación interna. Se utilizó parada temprana con restauración de los mejores pesos. Estas elecciones siguen el flujo de las clases 04 y 05.

Comparación sobre el mismo conjunto de prueba

Modelo	Exactitud	Precisión macro	Recall macro	F1 macro	Log loss	Entrenamiento
ANN	0.9713	0.9710	0.9711	0.9711	0.0971	5.17 s
CNN	0.9889	0.9888	0.9889	0.9888	0.0407	81.75 s

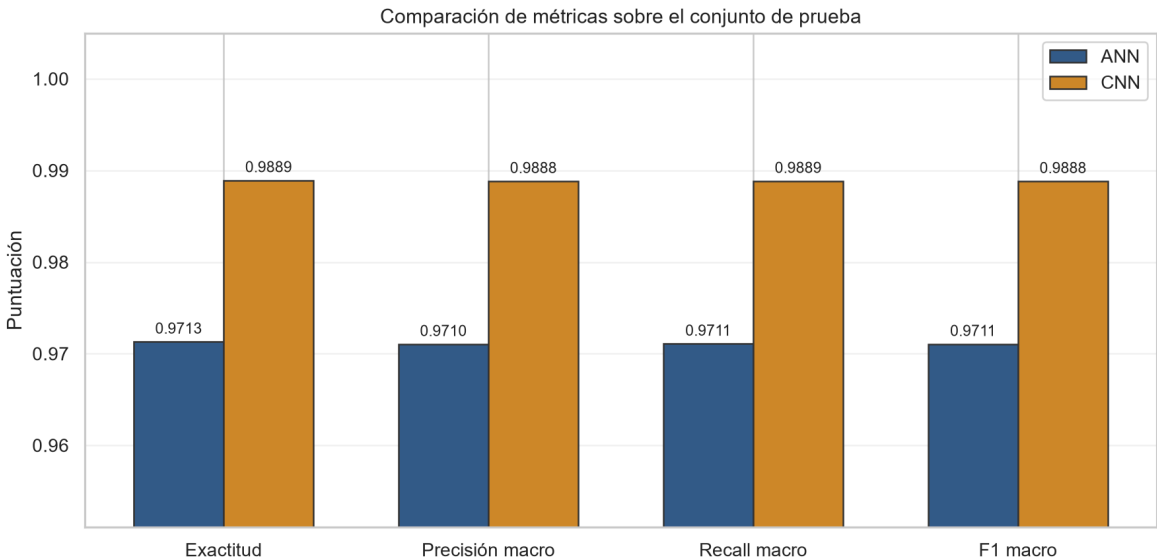


Figura 1. Métricas calculadas sobre 8,400 observaciones no utilizadas para ajustar los modelos.

Análisis, limitaciones y conclusión

+1.78 pp

ventaja de F1 macro

61.4%

menos errores de prueba

58.1%

menor pérdida logarítmica

15.8×

más tiempo de entrenamiento

Patrones de error

La mejora de la CNN no se limita a la exactitud: precisión, *recall* y F1 macro permanecen alrededor de 98.9%, lo que evidencia rendimiento equilibrado entre las diez clases. La ANN produjo 241 errores; sus confusiones más frecuentes fueron 4→9 (19 casos), 3→5 (17) y 5→3 (11). La CNN redujo el total a 93 errores; sus mayores confusiones individuales fueron 8→9 y 9→4 (seis casos cada una).

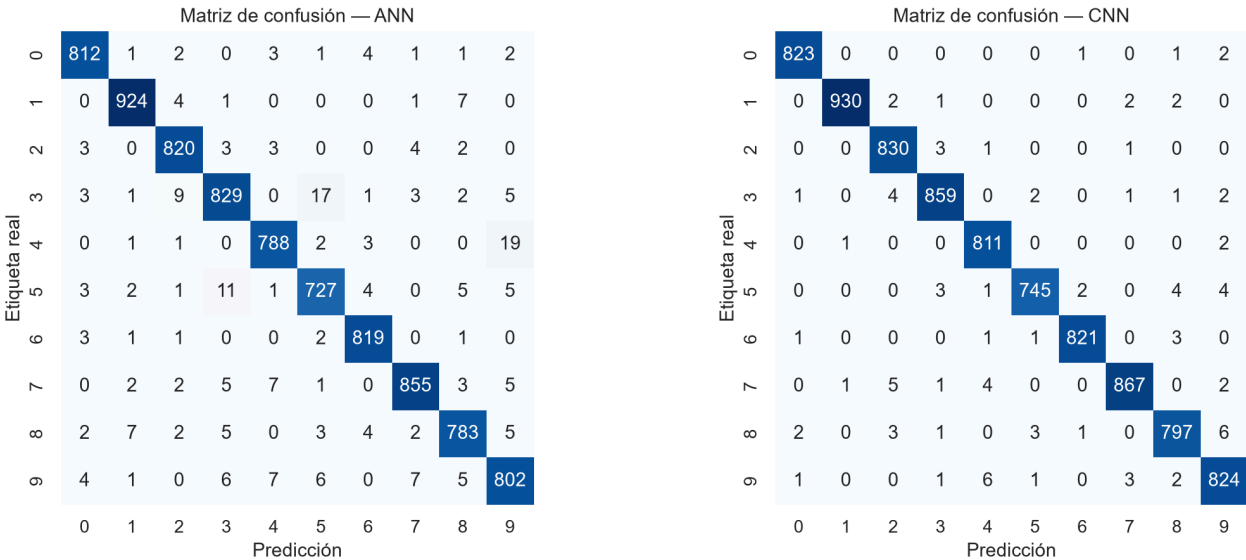


Figura 2. La mayor concentración en la diagonal de la CNN muestra una separación más consistente de los dígitos.

Interpretación

La CNN aprovecha patrones locales —bordes, trazos y combinaciones espaciales— mediante filtros aprendidos y conserva información que la ANN pierde al aplanar la imagen. El *max pooling* reduce dimensionalidad y retiene activaciones dominantes. Esto explica la mejora de 148 aciertos adicionales sobre el mismo conjunto de prueba.

La ganancia predictiva tiene un costo: la CNN utilizó 3.85 veces más parámetros y tardó 15.8 veces más en entrenarse en CPU. Sin embargo, su tiempo de inferencia para las 8,400 imágenes fue 0.58 s, adecuado para esta escala.

Conclusión

Se selecciona la CNN. Su exactitud de 98.89%, F1 macro de 98.88% y menor pérdida muestran una generalización superior para MNIST. El resultado respalda la hipótesis expuesta en la clase 05: conservar la geometría 28×28 y aprender características con convoluciones ofrece una ventaja real frente a una ANN que trabaja con los píxeles aplanados. La ANN sigue siendo una alternativa rápida y sólida (97.13%), pero la CNN es preferible cuando la prioridad es maximizar la calidad de clasificación.

Reproducibilidad. Python 3.12; TensorFlow 2.21; pandas 3.0.5; NumPy 2.5.2; scikit-learn 1.9.0. El notebook ejecutado contiene código, salidas, curvas, reportes por clase y análisis de errores. Referencias principales: *transcript clase 04.docx* (arquitectura CNN) y *transcript clase 05.docx* (instrucciones del Laboratorio No. 4), consultadas sin modificación.